

Symbols and Glossary

This glossary defines the Generalized Notation Notation (GNN) language constructs used to specify Active Inference generative models, together with the canonical Active Inference symbols those constructs carry. Definitions follow the GNN syntax specification and the discrete POMDP exemplars distributed with the repository, and they align with the standard discrete-state-space formulation of Active Inference (Smékal and Friedman 2023; Da Costa et al. 2020; Parr et al. 2022).

A GNN file is an ordered, UTF-8 Markdown document whose level-2 headers name required and optional sections. The strict schema validator enforces section order, declaration grammar, and connection syntax; the constructs below are the load-bearing sections a parser must recognize; they are catalogued in tbl. 1.

Language Constructs I

Table 1: The GNN language constructs a conforming parser must recognize, with the meaning each carries. The first six rows are the required sections that pin a model's identity, variables, and factor graph; the optional sections that follow carry parameterization, ontology bindings, timing, equations, and provenance. The three operator rows are the connection vocabulary the Connections section is written in — directed A>B, undirected A-B, and their labeled v1.1 forms — and default=... supplies initialization hints. Definitions follow the GNN syntax specification and the discrete POMDP exemplars under `input/gnn_files`, and the strict schema validator enforces section order and declaration grammar before any downstream step runs.

Construct	Meaning
## GNNSection	Required short identifier for the model block (no spaces, e.g. <code>ActInfPOMDP</code>).
## GNNVersionAndFlags	Required version declaration (<code>GNN v1</code> or <code>GNN v1.1</code>) with optional flags.
## ModelName	Required human-readable title of the generative model.
## StateSpaceBlock	Required block declaring every variable and matrix as <code>NAME[dim, ..., type=...]</code> , one per line.
## Connections	Required edge list relating state-space variables, expressing the model's factor graph.
## ModelAnnotation	Optional free-text description of the model's modalities, factors, and assumptions.
## InitialParameterization	Optional concrete numeric values for declared matrices and vectors.
## ActInfOntologyAnnotation	Optional bindings from each variable to a CamelCase Active Inference ontology term.
## ModelParameters	Optional key-value dimensions (e.g. <code>num_hidden_states</code> , <code>num_obs</code> , <code>num_actions</code>) consumed by code generators.
## Time	Optional dynamics declaration: a time variable plus <code>Dynamic/Static</code> , <code>Discrete/Continuous</code> , and <code>ModelTimeHorizon</code> .
## Equations	Optional LaTeX-rendered formulas defining model dynamics and the relationships between declared variables; round-tripped by the semantic-fidelity gate.
## Footer	Optional closing section that terminates the file and lets a reader or parser enter it from either end.

Language Constructs II

Signature

NAME[d ,d ,...,type=...]

A>B

A-B

A>B:label / A-B:label

default=...

Optional provenance block carrying a cryptographic signature over the specification.

A variable or tensor declaration; dimensions are positive integers or named references, and a `type` (`float`, `int`, `bool`) is required.

Directed (causal) connection operator: edge from A to B, e.g. `D>s` (a prior conditions a hidden state).

Undirected (bidirectional) connection operator, e.g. `s-A` (a hidden state participates in the likelihood mapping).

A v1.1 annotated edge; the trailing label documents the relation and is preserved but may be ignored for structural validation.

A v1.1 declaration hint (`uniform`, `zeros`, `ones`, `eye`, `random`) supplying an initialization for a matrix or vector.

The exemplar discrete POMDP agent declares the standard generative-model components of Active Inference over a discrete state space, mapping each GNN variable to its probabilistic meaning (Da Costa et al. 2020; Smith et al. 2022; Parr et al. 2022). Each row of tbl. 2 names the equation in sec. 3 that fixes the symbol's role, so the glossary and the formal statement cannot drift apart.

Active Inference Symbols I

Table 2: The Active Inference symbols a GNN specification carries, each bound to the equation of sec. 3 that fixes its role. Read A–E as the generative-model tensors (likelihood, controlled transitions, log-preferences, prior over initial states, habit prior), s , o , π , and u as the inference variables, and F and G as the two free-energy functionals minimized at eq. 6 and eq. 7. Use the table as a decoding key when reading a `StateSpaceBlock`: every declared matrix name should map to exactly one row here, and the `## ActInfOntologyAnnotation` bindings described below are what make that mapping machine-checkable downstream.

Symbol	Meaning
A	Likelihood (observation) matrix encoding $P(o s)$, mapping hidden states to observation outcomes (eq. 2).
B	Transition matrix encoding $P(s' s, u)$, mapping a previous state and action to the next state (eq. 3).
C	Preference vector: log-preferences over observation outcomes that bias the agent toward preferred outcomes (eq. 4).
D	Prior vector over initial hidden states (eq. 5).
E	Habit vector: an initial policy prior (baseline preference) over actions, entering the policy posterior (eq. 8).
s	Current hidden-state distribution; s_{prime} (s') is the next hidden-state distribution.
o	Current observation, an integer index over outcome modalities.
π	Policy: a distribution over actions inferred from expected free energy (eq. 8).
u	The selected (sampled) action, sampled from the policy posterior.
F	Variational free energy, minimized during state inference to update beliefs from observations (eq. 6) (Friston 2010).
G	Expected free energy per policy, minimized during policy inference to score candidate actions (eq. 7) (Da Costa et al. 2020).
t	Discrete time step; the horizon T bounds the product in eq. 1.

Ontology Bindings and Implementations I

The `## ActInfOntologyAnnotation` section binds each variable to a canonical term (`A=LikelihoodMatrix`, `B=TransitionMatrix`, `C=LogPreferenceVector`, `D=PriorOverHiddenStates`, `s=HiddenState`, `o=Observation`, π =`PolicyVector`), which downstream pipeline steps use for semantic analysis and validation. The same GNN specification feeds the project's rendering backends — including the 9 registered targets (`PyMDP`, `RxInfer.jl`, `ActiveInference.jl`, `JAX`, `DisCoPy`, `PyTorch`, `NumPyro`, `Stan`, `bnlearn`), of which the 9 listed as executable (`PyMDP`, `RxInfer.jl`, `ActiveInference.jl`, `JAX`, `DisCoPy`, `PyTorch`, `NumPyro`, `Stan`, `bnlearn`) also run at Step 12 — so that a model written once in this notation can be parsed, visualized, and executed across the 25-step

pipeline (0–24) without restating its mathematics (Smékal and Friedman 2023; Heins et al. 2022; Felice et al. 2021).